

JIT Simulation — Development Roadmap *Team Axora | Solo Build — Emmanuel*

MILESTONE 1 — Environment & Project Scaffold *You are here — nearly done*

What gets achieved:

- Virtual environment created and activated
- All packages installed and verified
- File structure in place (`app.py`, `jit_core.py`, `requirements.txt`)
- Git initialised and pushed to GitHub
- Streamlit runs successfully on localhost

Exit condition: `streamlit run app.py` opens in browser with no errors.

MILESTONE 2 — Core Physics & Logic Engine *File: `jit_core.py` — zero UI*

What gets achieved:

- `KalmanTracker` class built and working — takes noisy distance input, outputs smoothed distance and closing velocity
- `calculate_ttc_actual()` function — distance ÷ closing velocity, handles edge cases
- `calculate_ttc_min()` function — full formula with provisional `t_actuation` constant
- `FrictionEstimator` class — three-layer logic, simulation bypasses Layer 1 but class mirrors real architecture
- `JITStateMachine` class — all six states, all transitions, state history tracking
- `simulate_run()` function — takes inputs dict, runs 100 timesteps, returns all output arrays

Exit condition: You can run `jit_core.py` directly in terminal, call `simulate_run()` with a test input dict, and print the output arrays cleanly with no errors. No Streamlit involved yet.

MILESTONE 3 — Sidebar & Input Layer *File: `app.py` — inputs only, no charts yet*

What gets achieved:

- Streamlit sidebar fully built
- All six sliders wired up with correct ranges and defaults
- Rain toggle working
- RUN button and RESET button present
- Disabled camera button with caption rendered correctly

- Inputs collected into a clean dict ready to pass to `simulate_run()`
- App runs with no errors even though no outputs are shown yet

Exit condition: You can run the app, move all sliders, hit RUN, and see the input dict printed to terminal correctly via `st.write()` placeholder. No charts yet.

MILESTONE 4 — Status Cards & State Display *File: `app.py` — top row of main panel*

What gets achieved:

- Four `st.metric` cards rendering across the top of the main panel
- State card color-coded by current system state
- TTC Actual card showing computed value from `simulate_run()` output
- TTC Min card showing computed value
- Warning Stage card showing Stage 0, 1, or 2
- Cards update correctly when you change sliders and hit RUN

Exit condition: You run the simulation, change speed and friction, hit RUN again — all four cards update with correct values reflecting the new inputs.

MILESTONE 5 — Charts *File: `app.py` — main panel row 2*

What gets achieved:

- Distance over time chart — Plotly line chart, x-axis is timestep, y-axis is metres, line falls as obstacle closes
- TTC chart — two lines on same chart: `TTC_actual` (blue, falling) and `TTC_min` (red, flat horizontal). The crossing point is clearly visible and marks the trigger moment
- Both charts render side by side using `st.columns(2)`
- Charts update on every RUN press with new simulation data
- Axis labels, titles, and colors are clean and presentation-ready

Exit condition: You set high speed + low friction, hit RUN, and clearly see the blue TTC line cross the red `TTC_min` line on the chart. The crossing point is visually obvious to a non-technical judge.

MILESTONE 6 — Brake Gauge & State Timeline *File: `app.py` — main panel row 3*

What gets achieved:

- Brake engagement progress bar — shows 0% to max 30% pedal travel, only fills when state is ACTUATING
- State timeline horizontal bar chart — shows how many timesteps the system spent in each state during the run
- Both outputs render side by side using `st.columns(2)`
- Brake bar correctly stays at 0% for non-threatening scenarios and rises to ~30% when actuation fires

Exit condition: You run a high-threat scenario (high speed, low friction, high confidence) and see the brake bar fill. You run a low-threat scenario and see it stay at zero. State timeline reflects both correctly.

MILESTONE 7 — Degraded Camera Mode & Fallback Logic *Files: `jit_core.py` + `app.py`*

What gets achieved:

- Degraded camera mode triggers correctly when confidence slider is held below 0.2 for 3+ consecutive timesteps
- System state transitions to `DEGRADED_CAMERA` and shows correctly on state card
- Radar-only fallback rule fires when $TTC_actual < TTC_min \times 0.6$ in degraded mode
- Stage 1 indicator stays permanently lit in degraded mode
- All of this is visible and readable on the dashboard without explanation

Exit condition: You drag camera confidence to 0.1, set high speed, hit RUN — state card shows `DEGRADED_CAMERA`, Stage 1 lights up, and braking still fires via the radar fallback. You explain this to an imaginary judge in 30 seconds and they follow it.

MILESTONE 8 — Polish & Demo Hardening *Files: `app.py` + minor `jit_core.py` fixes*

What gets achieved:

- Team Axora branding in the app header
- JIT logo or wordmark if available
- Color scheme consistent and clean
- All edge cases handled — what if closing speed is zero, what if distance is already zero, what if friction is maximum
- RESET button fully clears all outputs and returns sliders to defaults
- App runs without any warnings or errors in the terminal
- Tested across the full competition demo flow from start to finish

Exit condition: You run the full demo flow — high speed, low friction, low confidence — from start to finish with no errors, no awkward moments, nothing to explain away. It just works.

MILESTONE 9 — Documentation & Handoff *Files: README.md + code comments*

What gets achieved:

- `README.md` written — what JIT is, how to run the simulation, what each input does, Team Axora credit
- Every function and class in `jit_core.py` has a clear docstring
- Inline comments explain non-obvious logic decisions (especially Kalman filter and degraded camera rule)
- GitHub repo is clean, committed, and shareable
- A second person could clone the repo, run `pip install -r requirements.txt`, and have it working in under 5 minutes

Exit condition: Abdulbasit or any team member clones the repo cold and runs it successfully without asking you a single question.

MILESTONE 10 — Competition Demo Recording *Output: screen recording + live demo ready*

What gets achieved:

- Full demo flow screen-recorded at high quality
- Narration script written — what to say at each moment of the demo
- Backup plan documented — if live demo fails, screen recording plays
- Simulation tested on the actual device that will be used for the presentation

Exit condition: You can demo this confidently in front of COUCH 2025 judges, explain every element on screen, and answer "what happens when the camera fails at night" without hesitation.

Summary

Milestone	Focus	File
1	Environment setup	—
2	Physics and logic engine	<code>jit_core.py</code>
3	Sidebar and inputs	<code>app.py</code>
4	Status cards	<code>app.py</code>

Milestone	Focus	File
5	Charts	app.py
6	Brake gauge and state timeline	app.py
7	Degraded camera mode	both
8	Polish and demo hardening	app.py
9	Documentation	README.md
10	Competition demo recording	—
